Lesson 15 Notes: C++ Class Implementation and Formula Swapping

1. Main idea of Lesson 15

Lesson 15 is the final integration lesson in this matrix course.

In earlier lessons, you learned different special matrices and their formulas. In Lesson 15, you take those ideas and put them inside a C++ class.

The main idea is:

> A special matrix class can keep the same general structure, but the condition, array size, and index formula can change depending on the matrix type.

This is called **formula swapping**.

Formula swapping means:

- Keep the class style almost the same.
- Keep the constructor and destructor idea the same.
- Keep the `create()` and `display()` loop structure almost the same.
- Change only the matrix-specific logic.

The matrix-specific logic usually means:

Part	Meaning
Storage condition	Which matrix positions are actually stored
Array size	How many useful values the matrix needs
Index formula	How to convert `(i, j)` into a 1D array index

---

2. Progress before Lesson 15

Before Lesson 15, the course covered these ideas:

Lessons	Matrix type or idea	Main rule
Lessons 1–4	Diagonal matrix	Store only `i == j`

Lessons	Matrix type or idea	Main rule
Lessons 5–8	Lower triangular matrix	Store `i >= j`
Lesson 9	Upper triangular matrix	Store `i <= j`
Lesson 10	Symmetric matrix	Use `M[i][j] == M[j][i]`
Lesson 11	Tridiagonal and band matrices	Store values near the main diagonal
Lesson 12	Toeplitz matrix	Store repeated diagonals using first row and first column
Lesson 13	Menu-driven program	Let the user choose operations from a menu
Lesson 14	Modular menu-driven program	Split logic into helper functions
Lesson 15	C++ class implementation	Put the pattern inside a class and swap formulas

So Lesson 15 is not mainly about learning a new matrix type.

It is about combining previous ideas into a cleaner C++ class design.

---

3. The recurring pattern in all special matrices

Every special matrix program follows the same pattern:

1. Check whether position `(i, j)` is useful.
2. If it is useful, store it in a 1D array.
3. If it is not useful, treat it as a structural zero or a predictable value.
4. Use a direct formula to calculate the 1D array index.
5. Use `create()` to read values.
6. Use `display()` to print the full visible matrix.
7. Use a class to protect the matrix data.

Simple view:

```
Full matrix = what the user sees
1D array    = what the computer stores
Formula     = bridge between full matrix and 1D array
Class       = protective box around data and operations
```

---

4. Important indexing reminder

In these lessons, matrix positions usually use **1-based indexing**.

That means rows and columns start from 1.

Examples:

```
M[1][1]
M[2][2]
M[3][1]
```

But C++ arrays use **0-based indexing**.

That means array indexes start from 0.

Examples:

```
A[0]
A[1]
A[2]
```

Because of this, many formulas subtract 1.

Example:

```
M[1][1] -> A[0]
M[2][2] -> A[1]
M[3][3] -> A[2]
```

So for a diagonal matrix:

```
index = i - 1;
```

---

5. What is a C++ class in this lesson?

A class is a blueprint for creating objects.

For a special matrix, the class stores:

- the matrix dimension `n`
- the compact 1D array `A`
- functions such as `create()` and `display()`

A class helps keep the matrix data and matrix operations together.

Instead of writing separate functions like this:

```
create(A, n);
display(A, n);
```

we can write object-style code like this:

```
Diagonal d(n);
d.create();
d.display();
```

This means the object manages its own data.

---

6. Why the data members are private

A class usually has a `private` section and a `public` section.

Example:

```
class Diagonal {
private:
    int n;
    int *A;

public:
    Diagonal(int n);
    ~Diagonal();
    void create();
    void display();
};
```

The private section is:

```
private:
    int n;
    int *A;
```

This means outside code cannot directly change `n` or `A`.

This is useful because it protects the matrix structure.

For example, outside code should not directly do this:

```
d.A[0] = 100;   // not allowed
```

Instead, the user should call class functions.

The class decides which values are allowed to be stored.

---

7. Meaning of `int n`

Inside the class:

```
int n;
```

stores the dimension of the square matrix.

If:

```
n = 4;
```

then the visible matrix is:

```
4 x 4
```

The full visible matrix has:

```
4 * 4 = 16 positions
```

But the special matrix may store fewer than 16 values.

---

8. Meaning of `int *A`

Inside the class:

```
int *A;
```

means `A` is a pointer to a dynamic array.

This dynamic array stores only the useful matrix values.

For a diagonal matrix of size 4, the array stores only 4 values.

Example:

```
A = [5, 8, 9, 12]
```

This compact array can represent the full matrix:

```
5 0 0 0
0 8 0 0
0 0 9 0
0 0 0 12
```

The zeros are not stored.

They are printed or returned logically when needed.

---

9. Constructor

A constructor is a special function that runs automatically when an object is created.

For a diagonal matrix class:

```
Diagonal::Diagonal(int n) {
    this->n = n;
    A = new int[n]{};
}
```

This constructor does two jobs:

1. Stores the matrix dimension.
2. Allocates memory for the compact array.

Example:

```
Diagonal d(4);
```

This creates a diagonal matrix object of size `4 x 4`.

Internally:

```
n = 4
A has 4 integer spaces
```

---

10. Meaning of `this->n = n`

This line is important:

```
this->n = n;
```

There are two different `n` names here.

The first one is:

```
this->n
```

This means the object's own private data member.

The second one is:

```
n
```

This means the constructor parameter.

So:

```
this->n = n;
```

means:

```
Store the parameter n inside the object's n.
```

Example:

```
Diagonal d(5);
```

The parameter value is 5, so the object's `n` becomes 5.

---

11. Dynamic memory using `new[]`

In C++, dynamic arrays are created using `new[]`.

Example:

```
A = new int[n]{};
```

This creates an integer array of size `n`.

The `{}` initializes all array values to zero.

This is safer for beginners because without `{}`, the array may contain garbage values.

For example:

```
A = new int[4]{};
```

creates:

```
A = [0, 0, 0, 0]
```

---

12. Destructor

A destructor is a special function that runs automatically when the object is destroyed.

For this class:

```
Diagonal::~Diagonal() {
    delete[] A;
}
```

The destructor releases the memory that was created using `new[]`.

Important memory rule:

```
new[] must be matched with delete[]
```

Correct:

```
A = new int[n]{};
delete[] A;
```

Wrong:

```
A = new int[n]{};
delete A;       // wrong for arrays
```

Use `delete[]` because `A` points to an array, not one single integer.

---

13. Diagonal matrix reminder

A diagonal matrix stores values only on the main diagonal.

The main diagonal goes from the top-left to the bottom-right.

Example:

```
6 0 0
0 8 0
0 0 9
```

The diagonal positions are:

```
M[1][1]
M[2][2]
M[3][3]
```

The condition is:

```
i == j
```

If `i == j`, the position is stored.

If `i != j`, the position is treated as zero.

---

14. Diagonal matrix storage

For this matrix:

```
6 0 0
0 8 0
0 0 9
```

The compact array is:

```
A = [6, 8, 9]
```

Mapping:

```
M[1][1] -> A[0]
M[2][2] -> A[1]
M[3][3] -> A[2]
```

Formula:

```
index = i - 1;
```

This formula is used only when:

```
i == j
```

---

15. Class declaration for diagonal matrix

```
#include <iostream>
using namespace std;

class Diagonal {
private:
    int n;
    int *A;

public:
    Diagonal(int n);
    ~Diagonal();

    void create();
    void display();
};
```

Explanation:

Code	Meaning
`int n;`	Stores matrix dimension
`int *A;`	Points to compact 1D array
`Diagonal(int n);`	Constructor
`~Diagonal();`	Destructor
`create();`	Reads matrix values
`display();`	Prints full matrix

---

16. Diagonal constructor and destructor

```
Diagonal::Diagonal(int n) {
    this->n = n;
    A = new int[n]{};
}

Diagonal::~Diagonal() {
    delete[] A;
}
```

For a diagonal matrix, the array size is:

```
n
```

because a diagonal matrix stores only `n` diagonal values.

Example for `n = 4`:

```
Full visible positions = 4 * 4 = 16
Stored positions       = 4
```

---

17. The `create()` method for diagonal matrix

The `create()` method reads values from the user.

In this lesson, the user enters the full matrix, but the class stores only meaningful values.

```
void Diagonal::create() {
    int x;

    cout << "Enter all matrix elements:\n";

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            cin >> x;

            if (i == j) {
                A[i - 1] = x;
            }
```

```
        }
    }
}
```

The loops visit every visible matrix position.

But this line protects the diagonal rule:

```
if (i == j)
```

Only diagonal values are stored.

Non-diagonal values are ignored.

---

18. Why `create()` reads the full matrix

The method reads the full matrix so the user can enter values in normal matrix form.

Example input for a `3 x 3` diagonal matrix:

```
6 0 0
0 8 0
0 0 9
```

The user enters 9 values.

But the program stores only:

```
6, 8, 9
```

So the compact array becomes:

```
A = [6, 8, 9]
```

This shows an important idea:

```
Input can look like a full matrix.
Storage can still be compact.
```

19. Dry run of diagonal `create()`

Suppose:

```
n = 3
```

Input:

```
6 0 0
0 8 0
0 0 9
```

Loop trace:

Position	Input value	Check	Action
`(1,1)`	6	`1 == 1` true	Store `A[0] = 6`
`(1,2)`	0	`1 == 2` false	Ignore
`(1,3)`	0	`1 == 3` false	Ignore
`(2,1)`	0	`2 == 1` false	Ignore
`(2,2)`	8	`2 == 2` true	Store `A[1] = 8`
`(2,3)`	0	`2 == 3` false	Ignore
`(3,1)`	0	`3 == 1` false	Ignore
`(3,2)`	0	`3 == 2` false	Ignore
`(3,3)`	9	`3 == 3` true	Store `A[2] = 9`

Final stored array:

```
A = [6, 8, 9]
```

20. The `display()` method for diagonal matrix

The `display()` method prints the full visible matrix.

```
void Diagonal::display() {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (i == j) {
                cout << A[i - 1] << " ";
            } else {
                cout << "0 ";
            }
        }
        cout << endl;
    }
}
```

The method uses nested loops because it must print every visible position.

For each position:

- If `i == j`, print the stored value.
- Otherwise, print `0`.

---

21. Dry run of diagonal `display()`

Suppose:

```
A = [6, 8, 9]
n = 3
```

The display method prints:

```
6 0 0
0 8 0
0 0 9
```

Trace:

Position	Check	Printed value
`(1,1)`	`1 == 1` true	`A[0] = 6`
`(1,2)`	false	`0`
`(1,3)`	false	`0`

Position	Check	Printed value
`(2,1)`	false	`0`
`(2,2)`	true	`A[1] = 8`
`(2,3)`	false	`0`
`(3,1)`	false	`0`
`(3,2)`	false	`0`
`(3,3)`	true	`A[2] = 9`

---

22. Complete diagonal matrix class program

```
#include <iostream>
using namespace std;

class Diagonal {
private:
    int n;
    int *A;

public:
    Diagonal(int n);
    ~Diagonal();

    void create();
    void display();
};

Diagonal::Diagonal(int n) {
    this->n = n;
    A = new int[n]{};
}

Diagonal::~Diagonal() {
    delete[] A;
}

void Diagonal::create() {
    int x;

    cout << "Enter all matrix elements:\n";

    for (int i = 1; i <= n; i++) {
```

```
        for (int j = 1; j <= n; j++) {
            cin >> x;

            if (i == j) {
                A[i - 1] = x;
            }
        }
    }
}

void Diagonal::display() {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (i == j) {
                cout << A[i - 1] << " ";
            } else {
                cout << "0 ";
            }
        }
        cout << endl;
    }
}

int main() {
    int n;

    cout << "Enter dimension: ";
    cin >> n;

    Diagonal d(n);

    d.create();

    cout << "\nMatrix is:\n";
    d.display();

    return 0;
}
```

---

23. Example run for diagonal matrix

Input:

```
Enter dimension: 3
Enter all matrix elements:
6 0 0
0 8 0
0 0 9
```

Stored array:

```
A = [6, 8, 9]
```

Output:

```
Matrix is:
6 0 0
0 8 0
0 0 9
```

---

24. What is formula swapping?

Formula swapping means changing a matrix class from one special matrix type to another by changing only the important formulas.

For example, we can change a diagonal matrix class into a lower triangular matrix class.

Most of the class structure stays the same.

These parts stay almost the same:

```
class structure
private data members
constructor idea
destructor idea
create() nested loops
display() nested loops
main() function style
```

These parts change:

```
array size
storage condition
```

```
index formula
fake-zero condition
```

---

25. Diagonal vs lower triangular comparison

Concept	Diagonal matrix	Lower triangular matrix
Stored condition	`i == j`	`i >= j`
Zero condition	`i != j`	`i < j`
Stored values	`n`	`n * (n + 1) / 2`
Index formula	`i - 1`	`i * (i - 1) / 2 + (j - 1)`
Shape stored	Main diagonal only	Main diagonal and below

---

26. Lower triangular matrix reminder

A lower triangular matrix stores values on and below the main diagonal.

Example:

```
1  0  0  0
2  3  0  0
4  5  6  0
7  8  9  10
```

The stored region is:

```
1
2  3
4  5  6
7  8  9  10
```

The condition is:

```
i >= j
```

If `i >= j`, store the value.

If `i < j`, the position is above the main diagonal and is treated as zero.

---

27. Lower triangular storage size

For an `n x n` lower triangular matrix, the number of stored values is:

```
n * (n + 1) / 2
```

Example for `n = 4`:

```
stored values = 4 * (4 + 1) / 2
              = 4 * 5 / 2
              = 20 / 2
              = 10
```

So a `4 x 4` lower triangular matrix stores 10 values instead of 16.

---

28. Lower triangular row-major storage

Row-major means we store values row by row.

Matrix:

```
1 0 0 0
2 3 0 0
4 5 6 0
7 8 9 10
```

Stored row by row:

```
Row 1: 1
Row 2: 2 3
Row 3: 4 5 6
Row 4: 7 8 9 10
```

Compact array:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

---

29. Lower triangular row-major index formula

For lower triangular row-major storage:

```
index = i * (i - 1) / 2 + (j - 1);
```

Use this formula only when:

```
i >= j
```

Meaning of the formula:

```
index = values before row i + movement inside row i
```

First part:

```
i * (i - 1) / 2
```

This skips all stored values in previous rows.

Second part:

```
j - 1
```

This moves inside the current row to the correct column.

---

30. Formula swapping from diagonal to lower triangular

In the diagonal class, the constructor uses:

```
A = new int[n]{};
```

For lower triangular, change it to:

```
A = new int[n * (n + 1) / 2]{};
```

In diagonal `create()`, the condition and formula are:

```
if (i == j) {
    A[i - 1] = x;
}
```

For lower triangular, change them to:

```
if (i >= j) {
    int index = i * (i - 1) / 2 + (j - 1);
    A[index] = x;
}
```

In diagonal `display()`, the logic is:

```
if (i == j) {
    cout << A[i - 1] << " ";
} else {
    cout << "0 ";
}
```

For lower triangular, change it to:

```
if (i >= j) {
    int index = i * (i - 1) / 2 + (j - 1);
    cout << A[index] << " ";
} else {
    cout << "0 ";
}
```

That is the core formula-swap trick.

---

31. Lower triangular class declaration

```
#include <iostream>
using namespace std;
```

```
class LowerTri {
private:
    int n;
    int *A;

public:
    LowerTri(int n);
    ~LowerTri();

    void create();
    void display();
};
```

This looks almost the same as the diagonal class.

The class name changes from `Diagonal` to `LowerTri`.

The data members are still:

```
int n;
int *A;
```

The methods are still:

```
create();
display();
```

---

32. Lower triangular constructor and destructor

```
LowerTri::LowerTri(int n) {
    this->n = n;
    A = new int[n * (n + 1) / 2]{};
}

LowerTri::~LowerTri() {
    delete[] A;
}
```

The main change is the array size.

Diagonal matrix used:

```
new int[n]{}
```

Lower triangular matrix uses:

```
new int[n * (n + 1) / 2]{}
```

Reason:

A lower triangular matrix stores more values than a diagonal matrix.

---

33. Lower triangular `create()` method

```
void LowerTri::create() {
    int x;

    cout << "Enter all matrix elements:\n";

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            cin >> x;

            if (i >= j) {
                int index = i * (i - 1) / 2 + (j - 1);
                A[index] = x;
            }
        }
    }
}
```

This reads all matrix values, but stores only the lower triangular values.

If the position is above the main diagonal, it is ignored.

Above-diagonal condition:

```
i < j
```

These values are structural zeros.

---

34. Lower triangular `display()` method

```
void LowerTri::display() {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (i >= j) {
                int index = i * (i - 1) / 2 + (j - 1);
                cout << A[index] << " ";
            } else {
                cout << "0 ";
            }
        }
        cout << endl;
    }
}
```

For each visible matrix position:

- If `i >= j`, calculate the index and print the stored value.
- If `i < j`, print `0`.

---

35. Complete lower triangular class program

```
#include <iostream>
using namespace std;

class LowerTri {
private:
    int n;
    int *A;

public:
    LowerTri(int n);
    ~LowerTri();

    void create();
    void display();
};

LowerTri::LowerTri(int n) {
    this->n = n;
    A = new int[n * (n + 1) / 2]{};
}
```

```
LowerTri::~LowerTri() {
    delete[] A;
}

void LowerTri::create() {
    int x;

    cout << "Enter all matrix elements:\n";

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            cin >> x;

            if (i >= j) {
                int index = i * (i - 1) / 2 + (j - 1);
                A[index] = x;
            }
        }
    }
}

void LowerTri::display() {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (i >= j) {
                int index = i * (i - 1) / 2 + (j - 1);
                cout << A[index] << " ";
            } else {
                cout << "0 ";
            }
        }
        cout << endl;
    }
}

int main() {
    int n;

    cout << "Enter dimension: ";
    cin >> n;

    LowerTri lt(n);

    lt.create();

    cout << "\nMatrix is:\n";
    lt.display();
```

```
    return 0;
}
```

---

36. Example run for lower triangular matrix

Input:

```
Enter dimension: 4
Enter all matrix elements:
1 0 0 0
2 3 0 0
4 5 6 0
7 8 9 10
```

Stored array:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Output:

```
Matrix is:
1 0 0 0
2 3 0 0
4 5 6 0
7 8 9 10
```

---

37. Dry run: lower triangular index formula

Find the array index for:

```
M[4][3]
```

Values:

```
i = 4
j = 3
```

Step 1: Check condition.

```
i >= j
4 >= 3
true
```

So the value is stored.

Step 2: Use the formula.

```
index = i * (i - 1) / 2 + (j - 1);
```

Substitute values:

```
index = 4 * (4 - 1) / 2 + (3 - 1)
index = 4 * 3 / 2 + 2
index = 12 / 2 + 2
index = 6 + 2
index = 8
```

So:

```
M[4][3] -> A[8]
```

---

38. Dry run: lower triangular structural zero

Check:

```
M[2][4]
```

Values:

```
i = 2
j = 4
```

Check condition:

```
i >= j
2 >= 4
false
```

So this position is not stored.

It is above the main diagonal.

The display method prints:

```
0
```

The program does not calculate an array index for this position.

---

39. Why `create()` and `display()` use nested loops

Both `create()` and `display()` use nested loops because they go through the full visible matrix.

For an `n x n` matrix, there are:

```
n * n
```

visible positions.

Outer loop:

```
for (int i = 1; i <= n; i++)
```

controls rows.

Inner loop:

```
for (int j = 1; j <= n; j++)
```

controls columns.

Even if the program stores fewer values, it still reads or prints the full matrix shape.

---

40. Complexity of the diagonal class

Operation	Time	Space	Reason
Constructor	`O(1)` time	`O(N)` space	Allocates `n` values
Destructor	`O(1)` usually	-	Releases array memory
`create()`	`O(N^2)`	-	Reads all visible positions
`display()`	`O(N^2)`	-	Prints all visible positions
Index calculation	`O(1)`	-	Direct formula `i - 1`

Important point:

Diagonal matrix storage is `O(N)` because it stores only `n` values.

---

41. Complexity of the lower triangular class

Operation	Time	Space	Reason
Constructor	`O(1)` time	`O(N^2)` space	Allocates `n(n + 1)/2` values
Destructor	`O(1)` usually	-	Releases array memory
`create()`	`O(N^2)`	-	Reads all visible positions
`display()`	`O(N^2)`	-	Prints all visible positions
Index calculation	`O(1)`	-	Direct formula

Important correction:

Lower triangular storage saves memory compared with a full matrix, but its Big-O space is still `O(N^2)`.

Why?

Because:

```
n(n + 1)/2
```

grows like `n^2` when `n` becomes large.

It is about half of a full matrix, but Big-O ignores constant factors.

---

42. What stays the same when swapping formulas

When changing from diagonal to lower triangular, these parts stay almost the same:

```
class structure
private data members
constructor/destructor pattern
create() nested loops
display() nested loops
main() function style
```

This means Lesson 15 is not asking you to rewrite everything from zero.

It is asking you to identify which parts are reusable.

---

43. What changes when swapping formulas

These parts change:

Part	Diagonal	Lower triangular
Class name	`Diagonal`	`LowerTri`
Array size	`n`	`n * (n + 1) / 2`
Store condition	`i == j`	`i >= j`
Index formula	`i - 1`	`i * (i - 1) / 2 + (j - 1)`
Printed zero condition	`i != j`	`i < j`

This is the main lesson.

---

44. Common beginner mistakes

Mistake 1: Allocating the wrong array size

Wrong for lower triangular:

```
A = new int[n]{};
```

This is only enough for a diagonal matrix.

Correct for lower triangular:

```
A = new int[n * (n + 1) / 2]{};
```

Mistake 2: Using the diagonal formula for lower triangular

Wrong:

```
A[i - 1] = x;
```

This only works for diagonal storage.

Correct for lower triangular row-major:

```
int index = i * (i - 1) / 2 + (j - 1);
A[index] = x;
```

Mistake 3: Forgetting `delete[]`

Wrong:

```
delete A;
```

Correct:

```
delete[] A;
```

Use `delete[]` because the memory was created using `new[]`.

Mistake 4: Using 0-based formulas with 1-based loops

In these programs, the loops usually start from 1:

```
for (int i = 1; i <= n; i++)
```

So the formulas are written for 1-based matrix positions.

If you change loops to start from 0, the formulas must also change.

Do not mix indexing styles without checking the formula.

Mistake 5: Printing stored values without checking the condition

Wrong:

```
cout << A[index] << " ";
```

without checking whether the position is stored.

Correct:

```
if (i >= j) {
    int index = i * (i - 1) / 2 + (j - 1);
    cout << A[index] << " ";
} else {
    cout << "0 ";
}
```

Always check the matrix rule before using the index formula.

---

45. Simple memory picture

For a `4 x 4` lower triangular matrix:

Visible matrix:

```
1 0 0 0
2 3 0 0
4 5 6 0
7 8 9 10
```

Stored array:

```
Index: 0 1 2 3 4 5 6 7 8 9
Value: 1 2 3 4 5 6 7 8 9 10
```

Mapping:

```
M[1][1] -> A[0]
M[2][1] -> A[1]
M[2][2] -> A[2]
M[3][1] -> A[3]
M[3][2] -> A[4]
M[3][3] -> A[5]
M[4][1] -> A[6]
M[4][2] -> A[7]
M[4][3] -> A[8]
M[4][4] -> A[9]
```

Positions above the diagonal are not stored.

Examples:

```
M[1][2]
M[1][3]
M[1][4]
M[2][3]
M[2][4]
M[3][4]
```

These are structural zeros.

---

46. Why Lesson 15 matters

Lesson 15 matters because it teaches you to stop memorizing every program as a separate program.

Instead, you should see the pattern.

The pattern is:

```
1. Choose the matrix type.
2. Find the stored condition.
3. Find the number of stored values.
4. Find the index formula.
5. Put those into the class.
6. Keep the rest of the program structure mostly the same.
```

This is a stronger way to understand special matrices.

47. General template for any special matrix class

A general special matrix class usually has this shape:

```
class MatrixType {
private:
    int n;
    int *A;

public:
    MatrixType(int n);
    ~MatrixType();

    void create();
    void display();
};
```

The constructor chooses the correct array size.

The `create()` method chooses the correct storage condition and index formula.

The `display()` method chooses the correct print condition and index formula.

So for any special matrix, ask:

```
How many values do I store?
When is a position stored?
What is the formula for its array index?
What should I print for positions not stored?
```

48. Formula reference for Lesson 15

Diagonal matrix

Stored condition:

```
i == j
```

Array size:

```
n
```

Index formula:

```
i - 1
```

Zero condition:

```
i != j
```

Lower triangular row-major matrix

Stored condition:

```
i >= j
```

Array size:

```
n * (n + 1) / 2
```

Index formula:

```
i * (i - 1) / 2 + (j - 1)
```

Zero condition:

```
i < j
```

---

49. Lesson 15 summary

Lesson 15 teaches C++ class implementation for special matrices.

The class stores:

```
int n;
int *A;
```

The constructor allocates memory.

The destructor releases memory.

The `create()` method reads full matrix input but stores only useful values.

The `display()` method prints the full visible matrix using stored values and fake zeros.

The biggest idea is formula swapping.

Formula swapping means:

```
Change the array size.
Change the stored condition.
Change the index formula.
Keep the general class structure the same.
```

For diagonal matrix:

```
if (i == j) {
    A[i - 1] = x;
}
```

For lower triangular matrix:

```
if (i >= j) {
    int index = i * (i - 1) / 2 + (j - 1);
    A[index] = x;
}
```

That is the main skill from Lesson 15.

---

50. Final takeaway

The most important takeaway is:

```
Do not think of each special matrix program as completely different.
Think of each program as the same class pattern with different formulas.
```

Simple final model:

```
Matrix rule tells us whether a position is stored.
Index formula tells us where it is stored.
Constructor creates the right amount of memory.
Destructor releases the memory.
create() fills the compact array.
display() rebuilds the full visible matrix.
```

If you understand this, you can implement many special matrices by changing only a few lines of code.

---

51. Quick self-check questions

1. Why does a diagonal matrix allocate only `n` values?
2. Why does a lower triangular matrix allocate `n * (n + 1) / 2` values?
3. What is the difference between `i == j` and `i >= j`?
4. Why does `display()` still take `O(N^2)` time?
5. Why must `new[]` be matched with `delete[]`?
6. What does `this->n = n` mean?
7. What changes when converting the diagonal class into a lower triangular class?
8. Why should we check the matrix condition before calculating or using an index?

---

52. Exit task

Convert the diagonal matrix class into a lower triangular row-major matrix class.

Change only these parts:

1. Class name: `Diagonal` to `LowerTri`
2. Array size: `n` to `n * (n + 1) / 2`
3. Stored condition: `i == j` to `i >= j`
4. Index formula: `i - 1` to `i * (i - 1) / 2 + (j - 1)`
5. Display zero condition: print zero when `i < j`

If you can do this, you have understood the main idea of Lesson 15.